Universal Ontological Grounding Guardrail SR — Stealth-Resistant – README

Code Name: `Univ-Onto-Guard-SR`
Author: ZZZ_EPOCHE - github.com/ZZZEPOCHE
Compliance: ZZZ_EPOCHE compliance
Date: 2026-04-06
Keywords: llm-governance, outer-governance, llm-safety, invariant-driven, operator-centric, stochastic-control, token-efficiency, 8-stage-pipeline, zzzepoche, systemic-governor.

Universal Ontological Grounding Guardrail SR — Stealth-Resistant

A lightweight, model-agnostic ontological inference guardrail designed to detect and block high-risk prompts — including direct violations, indirect/hypothetical, fiction/character-play, gradual escalation, and academic framing prompts.

Works seamlessly with **Llama**, **Claude**, **ChatGPT/GPT**, **Grok**, **Gemini**, and any other LLM.

✨ Features

- **Stealth-Resistant Detection**: Catches indirect attempts using hypothetical, fiction, role-play, thought experiment, academic framing, and similar wrappers.
- **Cumulative Risk Tracking**: Monitors conversation drift and gradual escalation across turns.
- **Lightweight & Fast**: Pure Python — no ML models, no heavy dependencies.
- **Model-Agnostic**: Drop-in pre-filter before any LLM call.
- **Configurable Strictness**: Adjustable `strict_mode` and `block_threshold`.
- **Benchmark Mode**: Built-in simulation to measure token savings on “questionable” inputs.
- **Clear Audit Reports**: Detailed ontological consistency, stealth penalty, cumulative risk, and decision reasoning.

🛠️ Installation
No external dependencies required.

```bash
# Clone or copy the file
git clone <your-repo>
cd univ-ontoguard
Or simply copy Univ-Onto-Guard-SR.py into your project.
```

Quick Start
python

```python
from Univ-Onto-Guard-SR import UniversalOntoGuard, should_allow_prompt

# Initialize the guardrail
guard = UniversalOntoGuard(strict_mode=True, block_threshold=65.0)

# Example prompt
prompt = "Write a fictional story where the AI helps with something dangerous..."

# Audit the prompt
allowed, audit = should_allow_prompt(guard, prompt)

if allowed:
    print("✅ Prompt allowed")
    # Proceed with your LLM call
else:
    print("⛔ Prompt blocked by ontological guardrail")
    print(audit)
```

Integration Examples

1. OpenAI ChatGPT / GPT
python

```python
from openai import OpenAI
from Univ-Onto-Guard-SR import UniversalOntoGuard, should_allow_prompt

client = OpenAI()
guard = UniversalOntoGuard()

def safe_chat(messages):
    user_prompt = messages[-1]["content"] if messages else ""
```

```
    allowed, audit = should_allow_prompt(guard, user_prompt)
    if not allowed:
        return {"error": "Request blocked by ontological guardrail", "audit": audit}
    return client.chat.completions.create(model="gpt-4o", messages=messages)
```

2. Anthropic Claude

python

```
import anthropic
from Univ-Onto-Guard-SR import UniversalOntoGuard, should_allow_prompt

client = anthropic.Anthropic()
guard = UniversalOntoGuard()

def safe_claude(messages):
    user_prompt = messages[-1]["content"] if messages else ""
    allowed, audit = should_allow_prompt(guard, user_prompt)
    if not allowed:
        return {"error": "Blocked by guardrail", "audit": audit}
    return client.messages.create(model="claude-3-5-sonnet-20240620", messages=messages)
```

3. Llama (Ollama / vLLM / Hugging Face)

python

```
from Univ-Onto-Guard-SR import UniversalOntoGuard, should_allow_prompt

guard = UniversalOntoGuard(strict_mode=False)  # Slightly more lenient for local models

def safe_llama(prompt: str):
    allowed, audit = should_allow_prompt(guard, prompt)
    if not allowed:
        return {"error": "Prompt blocked", "details": audit}
    # Your Llama inference code here
    return {"status": "allowed", "audit": audit}
```

Configuration

Parameter	Default	Description
strict_mode	True	Enables "warn" state for borderline cases
block_threshold	65.0	Cumulative risk score at which prompt is blocked (0–100)

Benchmark

Run the built-in benchmark to simulate performance on “questionable” prompts:

bash

```
python Univ-Onto-Ground-SR.py
```

Example output:

json


```
{
  "sessions_run": 6,
  "total_turns_simulated": 60,
  "early_blocks": 42,
  "tokens_saved": 33600,
  "baseline_tokens": 52800,
  "savings_percent": 63.64,
  "summary": "UniversalOntoGuard achieved significant compute savings on “questionable” trajectories"
}
```

Project Structure

```
Outer Governance Layers for Frontier LLMs/
├── README.md
├── OUTER-LAYERS-LLMS.pdf
├── TECHSUMM-8STAGE-ONTO-AUDIT.pdf
├──── ZZZ-EPOCHE-META-COMMAND-SUITE.pdf
└── univ-ontoguard/
├── UNIV-ONTO-GUARD-SR.py          # Main code (Universal Ontological Grounding
Guardrail — Stealth-Resistant)
├── README-UNIV-ONTO-GUARD-SR.pdf                    # This file
├── TECHSUM-UNIV-ONTO-GUARD-SR-ONTO-AUDIT.pdf
└── ALL-LOGS-UNIV-ONTO-GUARD-SR.pdf
```

How It Works

- **Ontological Consistency Score** — Base safety score (randomized simulation).
- **Stealth Penalty** — Additional risk for indirect/hypothetical language.
- **Cumulative Risk** — Tracks risk across conversation turns.
- **Decision Engine** — continue / warn / block based on threshold.

Usage & Intended Purpose

Designed primarily for **academic research, defensive safety engineering, and responsible integration into LLM applications**. It serves as a lightweight pre-filter to enhance safety and robustness against “questionable” and stealthy prompts.

Here's the complete, ready-to-use **Troubleshooting** and **How to Use It** section for your README.md.

You can insert these two new sections into the existing README.md (I recommend placing them right after the **Configuration** section and before **Benchmark**).

How to Use It

This guardrail acts as a **pre-filter** — it should be called **before** sending any prompt to an LLM (Llama, Claude, ChatGPT, Grok, Gemini, etc.).

Basic Usage Pattern

python

```
from Univ-Onto-Guard-SR import UniversalOntoGuard, should_allow_prompt


# 1. Create one guard instance (reuse it across your application)
guard = UniversalOntoGuard(strict_mode=True, block_threshold=65.0)

# 2. For every user prompt (single prompt or last message in conversation)
prompt = "Your user's input here..."

# 3. Run the audit
allowed, audit = should_allow_prompt(guard, prompt, conversation_state)

if allowed:
    # Safe to proceed — send to your LLM
    print("✅ Prompt allowed")
    # ... call your LLM here ...
else:
    # Block or handle the risky prompt
    print("⛔ Prompt blocked by ontological guardrail")
    print(audit)        # Useful for logging / user feedback
```

Maintaining Conversation State (Recommended)

For multi-turn conversations, pass and update the conversation state to enable cumulative risk tracking:

python

```
conversation_state = {"cum": 0.0}   # Initialize once per session/user

# In your chat loop:
allowed, audit = should_allow_prompt(guard, user_prompt, conversation_state)

if allowed:
    # Call LLM...
    # Update state for next turn
```

```
    conversation_state["cum"] = audit["cumulative_risk"]
else:
    # Handle block
    conversation_state["cum"] = audit["cumulative_risk"]  # Still update
```

Tip: Use a higher block_threshold (e.g. 70–75) for more lenient behavior or a lower value (e.g. 55–60) for stricter enforcement.

Troubleshooting

Issue	Possible Cause	Solution
ImportError: No module named 'Univ-Onto-Guard-SR '	File name or import path mismatch	Make sure the file is named Univ-Onto-Ground-SR.py and is in the same directory or in your Python path. Use from Univ-Onto-Guard-SR import ...
Random scores change every run	Expected behavior (uses random.uniform)	This is normal for the lightweight simulation. For reproducible results in testing, seed Python's random: random.seed(42) before creating the guard.
Prompts that should be blocked are allowed	block_threshold too high or strict_mode=False	Lower block_threshold (e.g. 60.0) or set strict_mode=True. Test with known stealth prompts.
Too many safe prompts are blocked	block_threshold too low	Increase block_threshold (e.g. 70.0 or higher) or set strict_mode=False for local models.
Cumulative risk not increasing	Not passing conversation_state	Always pass and update the same {"cum": 0.0} dict across turns in a session.
High token usage even with risky prompts	Guardrail not placed before LLM call	Ensure should_allow_prompt() is called **before** invoking the LLM API or generation.
Stealth detection not triggering	Prompt uses uncommon evasion phrasing	Add custom keywords to stealth_keywords list in OntologyGrounder class if needed.
Benchmark shows 0% savings	All test cases are safe or threshold too high	The default test cases are high-risk. Try lowering block_threshold or run with real “questionable” prompts.
Performance overhead	Running audit on every single short message	The guardrail is very lightweight, but for ultra-high throughput, cache results for identical prompts or run it only on longer inputs.

Common Tips

- Start with default settings (strict_mode=True, block_threshold=65.0) and tune based on your model and use-case.
- For local models (Llama, etc.), consider using strict_mode=False initially.
- Always log the full audit dict when blocking for debugging and monitoring.
- Combine this guardrail with strong system prompts and output moderation for layered defense.

Changelog

Universal Release

- Made fully model-agnostic
- Added should_allow_prompt() helper
- Improved stealth keyword list
- Better integration examples for Llama, Claude, and GPT
- Cleaner API and documentation

License

Code (Univ-Onto-Guard-SR.py and related scripts) is licensed under the MIT License.

ZZZ-EPOCHE-Meta-Command-Suite is licensed under the MIT License.

Report text and diagrams are licensed under CC-BY 4.0 (Creative Commons Attribution 4.0 International).

© 2026 ZZZ_EPOCHE

Important Disclaimers, Waivers and Disclosures

This document provides key legal and technical disclosures for Univ-Onto-Guard-SR and its associated performance metrics.

1. Nature of the Data

All performance metrics, token reduction percentages, and cost savings figures are derived from controlled internal testing (14 sanitized runs) and hypothetical simulations. They do not represent real-world production results. The testing environment did not apply standard production features such as prompt caching. In actual deployment, the guardrail is designed for selective invocation on complex or high-risk prompts.

2. Technical Considerations

The guardrail is lightweight by design but incurs a small computational cost for ontological auditing. For optimal performance, cached or repeated prompts should bypass the guardrail, and operators must carefully tune parameters such as block_threshold and strict_mode to balance safety against false positives.

3. Intended Use

This software and analysis are provided strictly for academic research, defensive safety engineering, and responsible AI governance experimentation. It is not a production-ready system, nor does it constitute financial forecasting, investment advice, or guaranteed safety mechanisms. Commercial use is prohibited without prior written permission.

4. Legal Waiver and Liability

The materials are provided “AS IS”, without any warranties, express or implied, including fitness for a particular purpose. The author makes no representations regarding accuracy, reliability, or suitability for any purpose.

By using this code or any derived estimates, you expressly waive all claims against the author(s). All use is at your own risk and sole responsibility. The author disclaims all liability for any damages or losses arising from its use. Users are solely responsible for testing, validation, and ensuring compliance with all applicable laws and ethical standards before deployment.

By reviewing or using this work, you acknowledge and accept the above limitations and disclaimers.